

EC2 → ECS Migration

멀쩡하게 쓰던 EC2를 ECS로 전환한 이야기

양승현 · 씨드투

오늘 이야기할 것

- ECS
- Migration 계기
- 고민과 걱정
- Migration 이후
- 시행착오
- 끝맺음

자기소개

- 양승현 / 백엔드 엔지니어 (라고 하지만 인프라 엔지니어링도 같이하고 있습니다)
- 어떻게 하면 딸각딸각할 것을 딸각 한번으로 줄일지 연구하고 실험하고 있습니다
- 일을 사서 만들고 있습니다.



01

ECS

ECS (Elastic Container Service)

AWS가 만든 컨테이너 오케스트레이션 서비스

- Cluster : Service와 Task를 묶는 최상위 논리적 Set
- Service : Task를 원하는 개수만큼 유지·관리하는 매니저
- Task : Container가 돌아가는 단위
- Task Definition : Task에 대한 설정

ECS (Elastic Container Service)

• Cluster 생성 화면

클러스터 생성 필수

Amazon ECS 클러스터는 태스크와 서비스를 함께 그룹화하고 용량과 공통적인 구성을 공유합니다. 모든 태스크, 서비스 및 용량은 클러스터에 속해 있습니다.

클러스터 구성

클러스터 이름

experienced-graffie-11u06

클러스터 이름은 1-255자여 합니다. 주요한 문자는 a-z, 0-9, 하이픈(-) 및 밑줄(_)입니다.

▼ 서비스 연결 기본값 - 선택 사항

기본 네트워크

네트워크를 선택할 수 있습니다. 기본 네트워크는 이 클러스터에 연결된 모든 서비스와 연결됩니다. 서비스 수준에서 이 값을 덮어쓸 수 있습니다.

Q. 새 네트워크 선택

새 네트워크 선택

▼ 인프라 - 고급 필수

에플리케이션을 호스팅하는 데 사용할 컴퓨팅 리소스를 확보하는 방법을 구성합니다.

컴퓨팅 용량 확보 방법 선택

클러스터는 AWS Fargate에 연결되어 있는 데 연결된 컴퓨팅 리소스를 추가하도록 선택할 수도 있습니다.

☒ Fargate 인스턴스

Fargate 인스턴스는 서버를 생성하거나 관리할 필요가 없습니다. 대부분의 일반적인 워크로드에 적합합니다.

☐ Fargate 및 자체 관리형 인스턴스

Fargate 및 자체 관리형 인스턴스는 Amazon ECS가 사용자 지정 인스턴스에 배치할 수 있는 워크로드를 관리하는 데 사용됩니다. 워크로드를 관리하는 데 필요한 리소스를 관리할 수 있습니다.

☐ Fargate 및 자체 관리형 인스턴스

Fargate 및 자체 관리형 인스턴스는 Amazon ECS가 사용자 지정 인스턴스에 배치할 수 있는 워크로드를 관리하는 데 사용됩니다. 워크로드를 관리하는 데 필요한 리소스를 관리할 수 있습니다.

▶ 모니터링 - 선택 사항

콘솔, 로그, 로그 그룹을 구성하여 컨테이너 환경의 구성 및 운영 가시성을 유지합니다.

▶ 암호화 - 선택 사항

이 클러스터에서 실행되는 태스크에서 스토리지를 암호화하는 데 사용되는 KMS 키를 선택합니다.

▼ 모니터링 - 선택 사항

콘솔, 로그, 로그 그룹을 구성하여 컨테이너 환경의 구성 및 운영 가시성을 유지합니다.

Container Insights 필수

CloudWatch 인사이트는 컨테이너에 대한 메트릭과 로그를 위한 모니터링 및 문제 해결 솔루션입니다.

Container Insights로 실행하고자 하는 컨테이너 수준을 선택하세요.

Container Insights는 모든 컨테이너 수준에서 실행될 수 있습니다.

☐ 향상된 관찰성을 갖춘 Container Insights 필수

Container Insights

클러스터 및 서비스 수준에서 집계된 데이터를 제공합니다. Logs Insights 뷰를 사용하여 실행 분석을 실행할 수 있습니다.

☒ 기본

기본 CloudWatch 지표만 제공합니다.

ECS Exec 암호화 및 로깅 필수

ECS Exec를 사용하여 컨테이너에서 명령을 실행하기 위한 로그 및 보안 설정을 구성합니다.

ECS 실행 세션용 KMS 키 ID

KMS 키 ID를 선택할 수 있습니다. 기본 클러스터에 연결된 컨테이너에 연결된 키를 선택합니다.

Q. AWS KMS 키를 선택하거나 ARN 입력

AWS KMS 키 선택

ECS Exec용 로깅

ECS Exec를 사용하여 실행되는 명령을 기록하기 위한 구성을 선택합니다.

☒ 기본값

테스크 정의에 포함된 awslogs 로그 드라이버를 사용하여 CloudWatch Logs의 로그를 전송합니다. 테스트 정의에 awslogs 로그 드라이버가 포함되어 있지 않으면 클러스터가 기본값을 사용합니다.

☐ 사용자 지정

CloudWatch Logs 로그 그룹, Amazon S3 버킷 또는 둘 다에 기록합니다. ECS 테스트 역할에는 CloudWatch 그룹/버킷의 IAM 권한이 포함되어 있습니다. 기본 AWS IAM 역할이 권한을 가진 경우 이 설정이 적용됩니다. 자세한 내용은 [이 페이지](#)를 참조하십시오.

☐ 없음

ECS Exec 실행 세션이 기록되지 않습니다.

▼ 암호화 - 선택 사항

이 클러스터에서 실행되는 태스크에서 스토리지를 암호화하는 데 사용되는 KMS 키를 선택합니다.

관리용 스토리지 필수

이 클러스터에서 실행되는 태스크에서 관리용 스토리지를 암호화하는 데 사용되는 기본 KMS 키를 선택합니다.

Q. AWS KMS 키를 선택하거나 ARN 입력

AWS KMS 키 선택

Fargate 임시 스토리지 필수

이 클러스터에서 실행되는 태스크에서 임시 스토리지를 암호화하는 데 사용되는 기본 KMS 키를 선택합니다. 자세한 내용은 [이 페이지](#)를 참조하십시오.

Q. AWS KMS 키를 선택하거나 ARN 입력

AWS KMS 키 선택

Fargate

- Task에 할당되는 “서버리스” 컴퓨팅 자원
- **AWS에서 관리**
- 키는 만큼 과금

CloudMap

- Fargate는 Task를 새로 띄울 때 마다 Private IP를 새로 생성
 - IP를 통한 네트워크연결은 무리
- 클라우드 리소스에 대해 알아보기 쉬운 이름을 정의해주는 서비스
- Task가 뜨고 죽을 때 현재 위치를 자동으로 갱신

02

Migration 계기

매번 똑같은 수작업

- EC2 한 대 띄울 때마다 콘솔에 들어가서 Docker 설치 / Docker Compose 설치 / 기본 폴더 구조
- 세팅 스크립트를 돌려도 간헐적으로 설정이 안 먹는 이슈
- 한두 대면 참는데… 대수가 늘어나면 답이 없음

OS 관리 부담

- 당시 OS는 Amazon Linux 2, 그런데 EOS(지원 종료) 안내 (2025.06)
- 보안 취약점(CVE) 대응하려면 → OS업데이트 or AMI 교체 필요
- 문제는 대수: 이걸 40대를 전부…?
- '이 관리 부담을 통째로 없앨 방법이 있을까?' → ECS(Fargate)

03

고민과 걱정

Cluster

- Cluster 구성을 어떻게 할까?
- Naming Convention 나중에 IaC/검색할 때 후회 안 하게 규칙부터

Service

- 이 Task를 몇 개로, 어떻게 유지할지' 정의하는 단위
- 하나의 Service에 어떤 Task Set 조합으로 둘지
(Gateway / Server 따로? Gateway & Server 합쳐서?)
- 배포 전략 설정(Rolling? Blue&Green?)
- CloudMap 설정(Gateway - Server 간의 네트워크 연결)

Task 구성(Task Definition)

- 실질적인 컨테이너의 설정
- CPU / Memory 를 얼마나 줄지
- IAM Role

Task Role : 이 Task가 어떤 AWS 권한을 가질지

Task Execution Role : 이 Task가 프로비저닝될 때 어떤 AWS 권한을 가질지

ex) ECR에서 이미지를 가져오는 권한, S3에서 설정파일을 가져오는 권한 등

비용

- ECS (Fargate, Task 1개 기준)
 - Gateway(0.25vCPU + 0.5GiB) : \$8.5 + \$1.86
 - Server(0.5vCPU + 1GiB) : \$16.99 + \$3.73
 - 합계 약 \$31.1
- EC2 (t3.small) : 약 \$19
 - 비용은 약 1.5배 증가

당시 이관대상 EC2 : 40개 => \$800

이관한다면 순수 유지비용 : $\$800 \times 1.5 = \1200

일단 띄워보자!

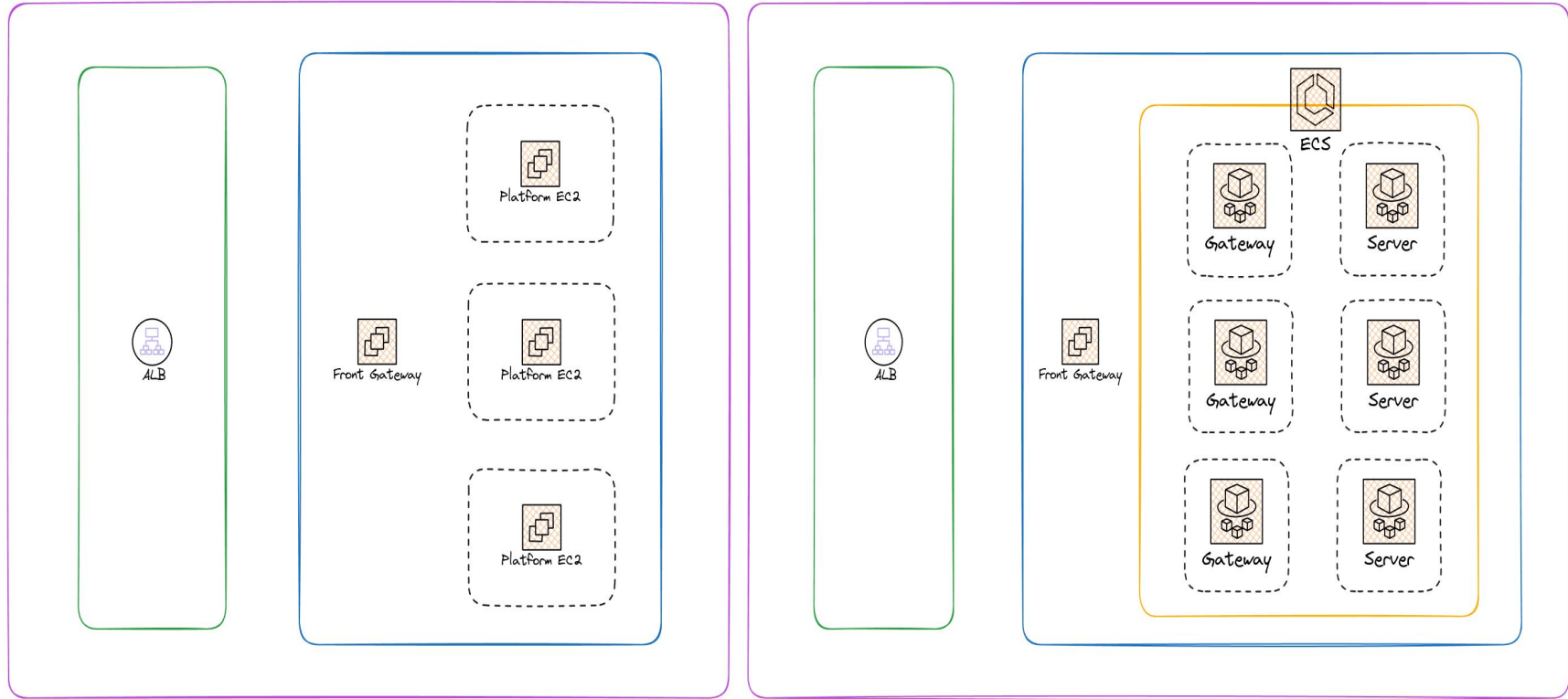


까짓거 한 번 해보죠

04

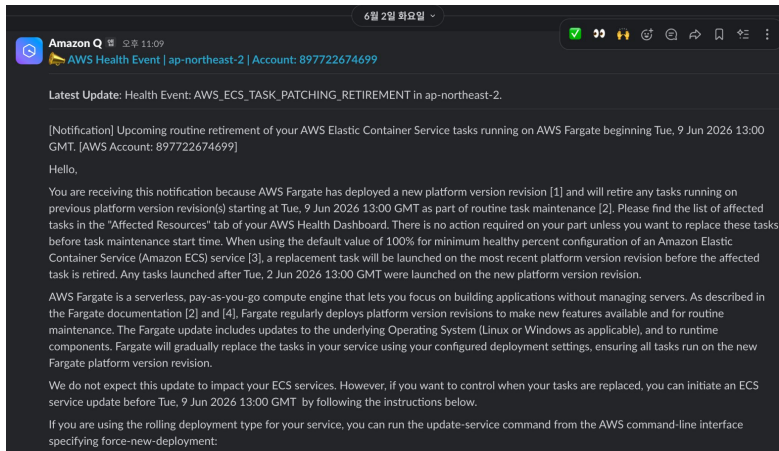
Migration 이후

Before VS After



관리 포인트 감소

- EC2 초기 세팅 생략 — Docker 깔고 폴더 잡던 반복 작업이 사라짐
- OS EOS 신경 끄 — AWS가 관리 (Fargate)
- 서버가 살았는지 죽었는지 터미널로 들어가서 확인할 필요가 없음



CloudWatch

- 이제 SSH로 서버에 들어가 로그를 직접 안 봐도 됨
- 컨테이너 로그가 CloudWatch로 모임 → 한 곳에서 검색/확인
- CloudWatch라는 지점으로 로그가 모이다보니 서버가 여러 대여도 로그가 흩어지지 않음

서버 알림 기능

EC2를 써도 알림 기능을 할 수는 있지만 WebApp, Container, EC2 이렇게 3개의 알림이 연관되서 만들어야해서 구현을 보류한 상황

ECS를 쓰니 EventBridge와 연동해서 Task 상태 변화를 감지 가능

- 서버(Task) 상태를 확실하게 확인할 수단이 생김

- Slack 알림까지 연결

05

시행착오

Service? Task? — 개념 혼동

- 처음에 이 둘의 차이를 이해하는 데 시간이 걸림
- Task = 실제로 도는 컨테이너 (일하는 사람)
- Service = Task를 몇 명 유지하고 어떻게 교체할지 관리 (관리자)

Cloud Map — 서비스끼리 찾아가기

- Gateway 라우팅 수정: Private IP 직접 연결 → Cloud Map host 이름으로 연결
- 인증 서버(EC2) · 결제 서버(EC2)로 가는 트래픽 설정
 - 이후 인증 서버 · 결제 서버도 차례로 ECS로 전환

IAM — 두 개의 Role 종류 혼동

- Task Role vs Task Execution Role
 - Task Role = 컨테이너 안 코드가 쓰는 권한 (직관적)
 - Task Execution Role = ECS가 컨테이너를 띄우기 위해 쓰는 권한(이미지 받기, 로그 쓰기)
- 배포 권한: 처음엔 GitHub Action OIDC를 위한 IAM Role
 - 이후 CodePipeline & CodeBuild로 이관

비용 — Container Insights 비용 폭탄

- Container Insights를 활성화했더니 비용이 급증
- 5일 뒤에야 발견해서 비활성화 처리
- 서비스 5개 기준, 5일 동안 \$430 발생

06

끝맺음

① 처음부터 ECS로 시작하진 않았다

- 처음엔 EC2로 시작
- 시작 단계라면 일단 EC2로 하고, 점진적으로 개선하는 걸 권장
- EC2로 먼저 굴러봤기에 ECS에서 필요한 자원(CPU/메모리 등)을 미리 알 수 있었음

② 이관은 신중하게

- 컴퓨팅 비용 1.5배는 결코 작은 수치가 아님
- 이관에는 여러 비용이 따름 → 이 작업이 어떤 가치를 주는지 확실히 잡고 갈 것
 - 방향성을 단단히 잡는 것 — ECS 이관에만 해당하는 얘기가 아님
- 체크포인트를 먼저 만들고 시작하자
 - 특히 IaC에 네이밍·설정값을 가지고 규칙을 만들 수 있기 때문에 더욱 중요

③ 일단 해보자

- 해보고 꼬였다 싶으면 지우고 다시 하면 됩니다. (백업은 필수)
- Task 실패 뜨고 롤백 나도 풀지 마세요
- 우리에겐 AI가 있다

감사합니다